AUSROS 2025
July, 2025
TurtleBot3 Challenge Task

1 Goals

Robots will be placed in an a priori unknown environment and must operate autonomously.

Single robot task: Explore and map an unknown environment in the fastest time.

Multi-robot adversarial task: Be the first to register the tag on the back of an opponent robot.

2 Setup

- Ubuntu 22.04 Jammy Jellyfish `https://releases.ubuntu.com/jammy/`
- ROS2 Humble Hawksbill `https://docs.ros.org/en/humble/index.html`
- TurtleBot3 Waffle Pi `https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/`
- Gazebo-11 (Simulation) `https://classic.gazebosim.org/`

3 Starter Kit

The following instructions are drawn from the Robotis TurtleBot3 e-manual, which can be found at `https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/`. Additional tools and examples are from the UQ METR4202 course, `https://github.com/METR4202`.

3.1 Gazebo Simulation

Simulation is a useful tool for rapid iteration and testing of your code before deployment on hardware. For this challenge, we will show you how to use Gazebo, a physics simulation with tight ROS2 integration and great support for the TurtleBot3 platform.

Gazebo-11 is already installed on the Ubuntu laptops with the basic libraries necessary for launching the TurtleBot3 in several simulation environments. If you would like to set up your own device in the same way, please see `https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/`.

To launch a TurtleBot3 WafflePi in Gazebo:

```
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Other pre-built Gazebo environments can be found within the `turtlebot3_gazebo` package, or in `https://github.com/METR4202/metr4202_aruco_explore`.

3.2 TurtleBot3 Bringup

Make sure both the Ubuntu laptop and the TurtleBot3 are on the same wifi network.

1. Launch `bringup` on the TurtleBot3. From the Ubuntu laptop:

   ```
   ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}
   export TURTLEBOT3_MODEL=waffle_pi
   ros2 launch turtlebot3_bringup robot.launch.py
   ```

2. If you are using the camera for your task, launch the camera node on the TurtleBot3. From the Ubuntu laptop:

   ```
   ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}
   ros2 run v4l2_camera v4l2_camera_node
   ```

3. (TODO) Check if images need to be compressed and/or change image resolution settings on camera (see note at the bottom of page `https://emanual.robotis.com/docs/en/platform/turtlebot3/sbc_setup/`

3.3 TurtleBot3 Navigation

The `turtlebot3_navigation` package provides an interface to the ROS2 Nav2 library (`https://docs.nav2.org`), which has implementations for mapping, localisation, SLAM, and waypoint navigation.

To run SLAM and waypoint navigation:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py slam:=True
```

In addition, if running in simulation, include the flag `use_sim_time:=True`

3.4 Waypoint Publishing Node

1. Create a new workspace on the Ubuntu laptop:

   ```
   cd
   mkdir -p ausros_ws/src
   cd ausros_ws
   colcon build
   ```

2. Source the workspace

   ```
   source ~/ausros_ws/install/setup.bash
   ```

   Note that you can include this line in the `~/.bashrc` file so that the workspace is sourced whenever a new terminal is opened

   ```
   echo 'source ~/ausros_ws/install/setup.bash' >> ~/.bashrc
   ```

3. Create a new package in your workspace called `waypoint_commander`

   ```
   cd /path/to/your/ausros_ws/src
   ros2 pkg create --build-type ament_python waypoint_commander --dependencies rclpy
   ↪  nav2_msgs geometry_msgs
   ```

   You can also manually update the package's execution time dependencies after you've created the package by including the relevant lines in `package.xml`, e.g.

   ```
   <exec_depend>rclpy</exec_depend>
   <exec_depend>nav2_msgs</exec_depend>
   <exec_depend>geometry_msgs</exec_depend>
   ```

4. Create a new python file called `waypoint_cycler.py` in your package. You can do this through the command line using the following:

```
cd /path/to/your/ausros_ws/src/waypoint_commander/waypoint_commander
touch waypoint_cycler.py
```

5. Open your new python file in VS Code (or your preferred editor) and begin by creating the main function call interface and importing the relevant libraries.

```python
import rclpy
from rclpy.node import Node

from nav2_msgs.msg import BehaviorTreeLog
from geometry_msgs.msg import PoseStamped


def main(args=None):
    rclpy.init(args=args)

    # Create a new WaypointCycler node
    waypoint_cmder = WaypointCycler()

    # Execute the node
    rclpy.spin(waypoint_cmder)


if __name__ == '__main__':
    main()
```

6. Create the `WaypointCycler` node class and initialise it with four variables:

   i) a subscriber to the `behavior_tree_log` topic (note the US spelling)

   ii) a publisher that publishes to the `goal_pose` topic

   iii) a waypoint counter

   iv) a list of waypoints

```python
class WaypointCycler(Node):

    def __init__(self):
        super().__init__('waypoint_cycler') # this defines the node name

        # Create a subscriber to the behavior_tree_log topic
        # The constructor inputs are (message type, topic name, associated
        ↪  callback function, message queue length)
        self.subscription = self.create_subscription(
            BehaviorTreeLog,
            'behavior_tree_log',
            self.bt_log_callback,
            10)
        self.subscription  # prevent unused variable warning

        # Create a publisher
        # The constructor inputs are (message type, topic name, message queue
        ↪  length)
        self.publisher_ = self.create_publisher(
            PoseStamped,
            'goal_pose',
            10)

```

```
        # Keep track of the number of waypoints sent
        self.waypoint_counter = 0

        # Manually input the two waypoints
        # Best practice would be to put these in a config file that's read in as params)
        # Note also that these waypoints have only been tested for the TurtleBot3 Gazebo environment
        p0 = PoseStamped()
        p0.header.frame_id = 'map'
        p0.pose.position.x = 1.7
        p0.pose.position.y = -0.5
        p0.pose.orientation.w = 1.0

        p1 = PoseStamped()
        p1.header.frame_id = 'map'
        p1.pose.position.x = -0.6
        p1.pose.position.y = 1.8
        p1.pose.orientation.w = 1.0

        # Store the waypoints as a member variable
        self.waypoints = [p0, p1]
```

7. The `behavior_tree_log` topic publishes the navigation action status. For our purposes, we need to check when the top-level planning node (`NavigateRecovery`) becomes `IDLE`, this indicates when the robot has either successfully reached its goal pose, or has experienced a planning or execution failure.

   We access this information through the callback function that we declared in the class constructor (`__init__`). We can define it as below:

```
    def bt_log_callback(self, msg:BehaviorTreeLog):
        # Check the current state of the behaviour tree
        # When the robot has reached its waypoint, the top level node state is 'NavigateRecovery' and the event status is 'IDLE'
        # If the behaviour tree is in this state, send the next waypoint
        for event in msg.event_log:
            if event.node_name == 'NavigateRecovery' and event.current_status == 'IDLE':
                self.send_waypoint()
```

8. Now we need to define the function `send_waypoint` so that we actually send it the next waypoint. We can cycle through the two waypoints that we manually defined in the constructor by using a mod 2 operation.

```
    def send_waypoint(self):
        # Keep track of the number of waypoints sent
        self.waypoint_counter += 1

        # Mod operation to determine if waypoint count is odd or even
        if self.waypoint_counter % 2:
            self.publisher_.publish(self.waypoints[1])
        else:
            self.publisher_.publish(self.waypoints[0])
```

9. Save your file and open the `setup.py` file in your top-level `waypoint_commander` folder. In order for ROS to know about our new node, we need to include the following after line 22 (inside the square brackets):

```
'waypoint_cycler = waypoint_commander.waypoint_cycler:main',
```

10. In a terminal, colcon build your package from your workspace folder.

```
colcon build --symlink-install --packages-select waypoint_commander
```

11. Run your node!

    ```
    ros2 run waypoint_commander waypoint_cycler
    ```

    To kick off `waypoint_cycler` you need to first send a manual waypoint to the TurtleBot3 (either in RViz or through the command line). Once it reaches this waypoint, it will begin cycling through the two manually defined waypoints.